

# CSE422 - Genetic Algorithm

## 1 Genetic Algorithms (GAs) - Concept

**Genetic algorithms (GAs)** are a class of **optimization algorithms** inspired by the principles of **natural selection and genetics**. These algorithms are used to solve **search and optimization problems** and are particularly effective for **complex issues** that are difficult to solve using **traditional methods**.

Genetic algorithms mimic the process of natural evolution, embodying the survival of the fittest among possible solutions. The core idea is derived from biological mechanisms such as **reproduction, mutation, recombination, and selection**. These concepts are translated into computational steps that help find optimal or near-optimal solutions across engineering, economics, and artificial intelligence.

### 1.1 Global Best vs. Local Best

Genetic algorithms evaluate large search spaces for a good solution. A GA is **not guaranteed** to find the absolute best solution; it attempts to find the **global best** while avoiding getting trapped at **local best** solutions.

- **Global best:** best possible solution in the entire space.
- **Local best:** best in a neighborhood but not the global optimum.
- If **smaller is better**  $\Rightarrow$  minimize. If **larger is better**  $\Rightarrow$  maximize.

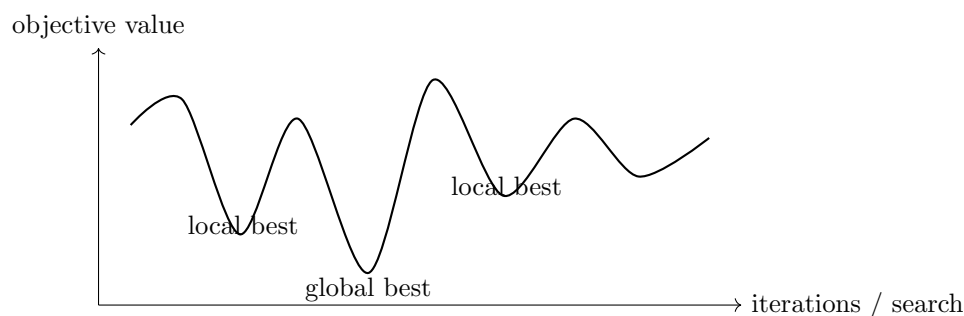


Figure 1: Illustration: local best vs. global best.

### 1.2 General Life Cycle of a Genetic Algorithm

- **Creating a population** - Create a random population of potential solutions.
- **Measuring fitness** - Score each solution using a **fitness function**.
- **Selecting parents** - Prefer higher-fitness individuals as parents.

- **Reproducing offspring** - Apply crossover and small random mutations.
- **Populating next generation** - Keep best/offspring to form new population.

## 2 1. Problem Overview (VLSI Floorplanning)

We simulate a simplified version of **VLSI floorplanning** within a chip circuit. Goal: determine an optimal placement of **6 components** on a 2D chip grid, minimizing:

1. total **wire length** between required connected pairs,
2. total required **chip area** (bounding box area),
3. and **avoid overlapping** components (heavily penalized).

We design a GA-based optimizer for this.

## 3 2. Grid and Components

### 3.1 Chip Grid

Assume a chip grid of dimension  $25 \times 25$  unit squares (you may later vary the grid size as an input parameter).

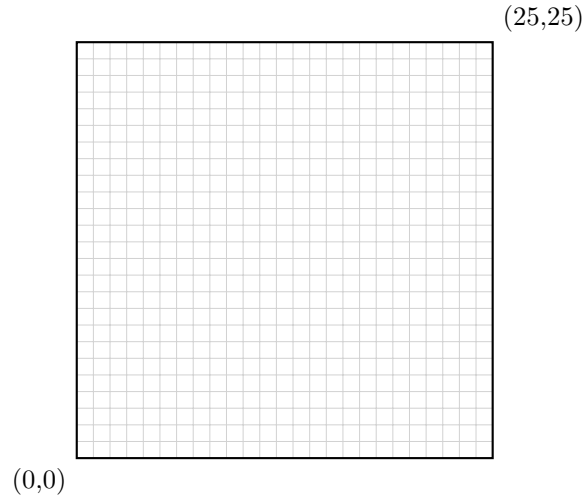


Figure 2:  $25 \times 25$  chip grid.

### 3.2 Component Blocks (Fixed Sizes)

| Block Name    | Width (unit) | Height (unit) |
|---------------|--------------|---------------|
| ALU           | 5            | 5             |
| Cache         | 7            | 4             |
| Control Unit  | 4            | 4             |
| Register File | 6            | 6             |
| Decoder       | 5            | 3             |
| Floating Unit | 5            | 5             |

Table 1: 6 functional blocks and their fixed dimensions.

### 4 3. Required Interconnections (6 pairs)

We minimize the wiring distance between these **6 connected component pairs**:

- Register File  $\rightarrow$  ALU
- Control Unit  $\rightarrow$  ALU
- ALU  $\rightarrow$  Cache
- Register File  $\rightarrow$  Floating Unit
- Cache  $\rightarrow$  Decoder
- Decoder  $\rightarrow$  Floating Unit

Assume bidirectional connections, no extra area, and wires run **center-to-center**.

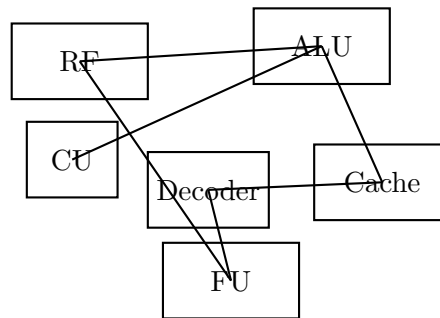


Figure 3: Required connectivity graph (schematic).

## 5 4. Fitness Objectives

We design a fitness function that reflects three objectives:

1. **Minimize overlaps** (highest priority; heavy penalty).
2. **Minimize total wiring distance**.
3. **Minimize bounding box area**.

### 5.1 4.1 Minimizing Total Wiring Distance (center-to-center)

#### 5.1.1 Step A: Convert bottom-left coordinate to center

For a block with bottom-left  $(X, Y)$  and dimensions  $(W, H)$ :

$$X_c = X + \frac{W}{2}, \quad Y_c = Y + \frac{H}{2}.$$

#### 5.1.2 Step B: Distance between two component centers

For two centers  $(x_1, y_1)$  and  $(x_2, y_2)$ , Euclidean distance:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

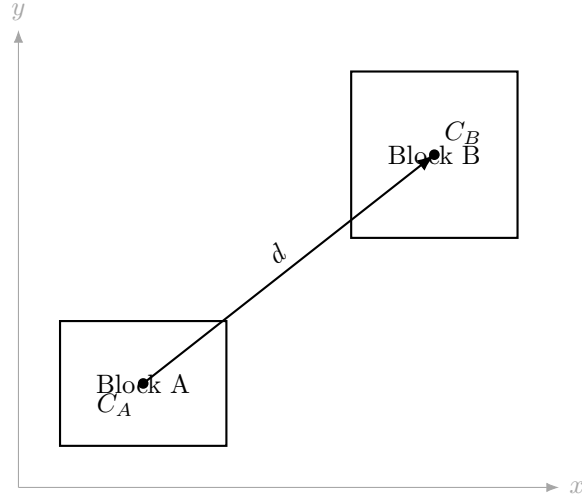


Figure 4: Center-to-center Euclidean wiring distance.

### 5.2 4.2 Minimizing Total Bounding Box Area

The bounding box is the smallest rectangle that contains **all** blocks.

For each block  $i$  with bottom-left  $(x_i, y_i)$  and size  $(w_i, h_i)$ :

$$x_{\min} = \min_i x_i, \quad x_{\max} = \max_i (x_i + w_i),$$

$$y_{\min} = \min_i y_i, \quad y_{\max} = \max_i (y_i + h_i).$$

Bounding box area:

$$A_{\text{bbox}} = (x_{\max} - x_{\min}) (y_{\max} - y_{\min}).$$

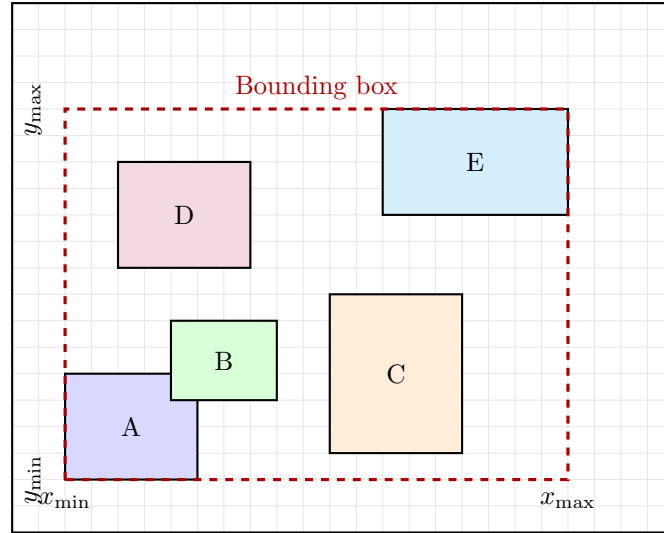


Figure 5: Bounding box (dashed red) for a set of blocks.

### 5.3 4.3 Minimizing Total Component Overlaps (heavy penalty)

#### 5.3.1 Overlap condition (rectangle separation test)

For rectangles  $A$  and  $B$ :

$$\text{NoOverlap} \iff (A_{\text{right}} \leq B_{\text{left}}) \text{ or } (A_{\text{left}} \geq B_{\text{right}}) \text{ or } (A_{\text{bottom}} \geq B_{\text{top}}) \text{ or } (A_{\text{top}} \leq B_{\text{bottom}}).$$

So,

$$\text{Overlap} \iff \neg(\text{NoOverlap}).$$

**Meaning:**

- If **any** separation case is true  $\Rightarrow$  rectangles are separated  $\Rightarrow$  **no overlap**.
- Overlap exists only if **none** of the separation cases are true.
- Two rectangles overlap if:
  - $A$  is **not completely left** of  $B$ ,
  - $A$  is **not completely right** of  $B$ ,
  - $A$  is **not completely above** of  $B$ ,
  - $A$  is **not completely below** of  $B$ .

#### 5.3.2 Overlap Example (bottom-left coordinates; plot points + rectangles)

Example rectangles:

- Block A: bottom-left (2, 3), top-right (7, 7)
- Block B: bottom-left (6, 5), top-right (10, 8)

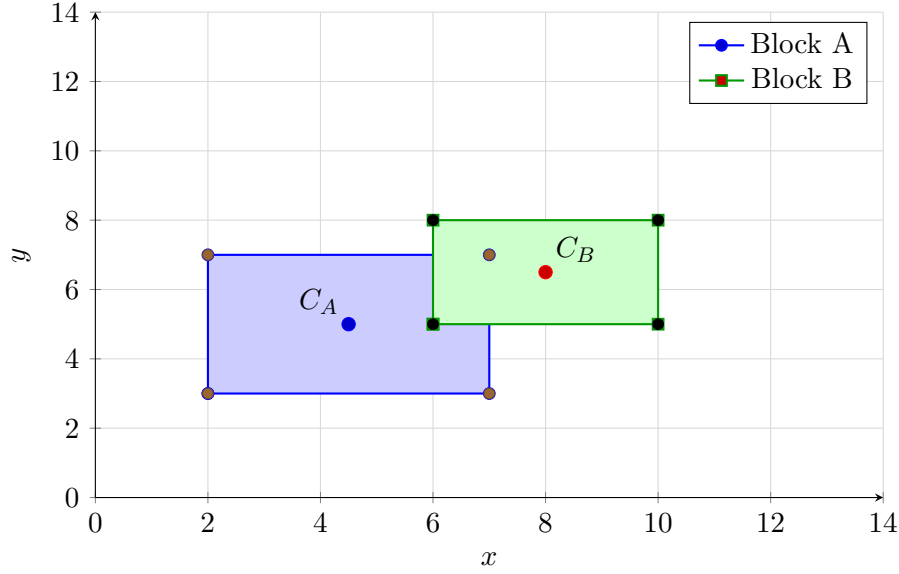


Figure 6: Overlap example: plotted corner points + rectangles + centers.

## 6 Task 1 Instructions

### 6.1 (1) Chromosome Representation / Encoding

Start with a random initial population of 6 members. Each chromosome represents a candidate layout.

The layout is specified by all the **bottom-left coordinates** of the 6 components (in Section 2 order):

(ALU), (Cache), (Control Unit), (Register File), (Decoder), (Floating Unit).

Coordinate values are in  $[0, 25]$ .

- Devise a suitable encoding scheme to represent all components (avoid redundancy).
- Generate an initial population of 6 chromosomes.

### 6.2 Sample Initial Population (6 chromosomes)

Each line represents bottom-left positions of the 6 blocks in the exact order above:

$P1 \rightarrow (9, 3), (12, 15), (13, 16), (1, 13), (4, 15), (9, 6)$

$P2 \rightarrow (8, 0), (7, 12), (4, 11), (1, 13), (14, 10), (9, 11)$

$P3 \rightarrow (6, 5), (12, 9), (9, 7), (8, 6), (2, 7), (3, 1)$

$P4 \rightarrow (3, 11), (11, 12), (14, 11), (6, 10), (3, 11), (3, 0)$

$P5 \rightarrow (10, 12), (8, 16), (10, 4), (13, 6), (6, 0), (3, 7)$

$P6 \rightarrow (0, 2), (0, 0), (14, 12), (4, 5), (12, 4), (3, 10)$

### 6.3 (2) Fitness Function Implementation

Design a fitness function based on the 3 objectives:

- Overlap counts: **least desirable**  $\Rightarrow$  penalize much more.

2. Total wiring distance and bounding box area: next priorities.

A weighted sum (used in the note):

$$\text{Total fitness} = -(\alpha \cdot \text{overlap} + \beta \cdot \text{wiring} + \gamma \cdot \text{area}),$$

with:

$$\alpha = 1000, \quad \beta = 2, \quad \gamma = 1.$$

#### 6.4 Sample Fitness Table (given)

| Chromosome | Overlap | Wiring | Area | Fitness  |
|------------|---------|--------|------|----------|
| P1         | 3       | 67.1   | 306  | -3440.23 |
| P2         | 3       | 57.7   | 342  | -3457.40 |
| P3         | 7       | 37.8   | 204  | -7279.63 |
| P4         | 5       | 53.3   | 240  | -5346.51 |
| P5         | 2       | 54.5   | 320  | -2429.00 |
| P6         | 3       | 52.6   | 288  | -3393.18 |

Table 2: Example computed fitness values.

#### 6.5 Pairwise overlap counting detail

There are 6 components. Pairwise overlap checks:

$$\binom{6}{2} = 15.$$

So, for each chromosome, check overlaps across the 15 unordered pairs and count how many overlap.

## 7 Section 7: Detailed Sample Input/Output Computation for $P1$

We now **recompute everything step-by-step** for  $P1$  using:

$$P1 \rightarrow (9, 3), (12, 15), (13, 16), (1, 13), (4, 15), (9, 6)$$

in the order (ALU, Cache, CU, RF, Decoder, FU) and using the fixed sizes table.

### 7.1 Step 0: Expand each block to its rectangle corners

For each block: bottom-left  $(x, y)$  and top-right  $(x + w, y + h)$ .

| Block | $(x, y)$ | $(w, h)$ | Top-right $(x + w, y + h)$ | Center $(x + \frac{w}{2}, y + \frac{h}{2})$ |
|-------|----------|----------|----------------------------|---------------------------------------------|
| ALU   | (9,3)    | (5,5)    | (14,8)                     | (11.5, 5.5)                                 |
| Cache | (12,15)  | (7,4)    | (19,19)                    | (15.5, 17.0)                                |
| CU    | (13,16)  | (4,4)    | (17,20)                    | (15.0, 18.0)                                |
| RF    | (1,13)   | (6,6)    | (7,19)                     | (4.0, 16.0)                                 |
| Dec   | (4,15)   | (5,3)    | (9,18)                     | (6.5, 16.5)                                 |
| FU    | (9,6)    | (5,5)    | (14,11)                    | (11.5, 8.5)                                 |

### 7.2 Step 1: Plot $P1$ on the $25 \times 25$ grid (rectangles + centers + wires)

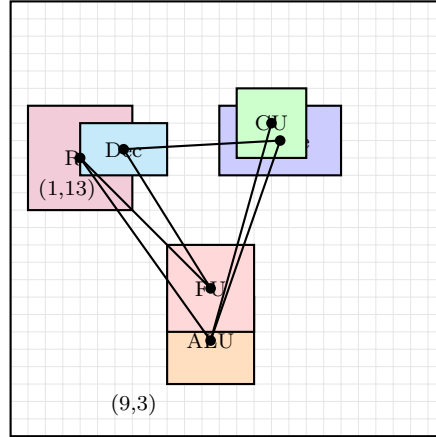


Figure 7:  $P1$  layout: rectangles + plotted center points + required 6 wires.

### 7.3 Step 2: Total wiring distance of the 6 required pairs (full calculation)

We compute each distance using:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}.$$

(i) **RF**  $\rightarrow$  **ALU**

Centers:

$$C_{\text{RF}} = (4, 16), \quad C_{\text{ALU}} = (11.5, 5.5).$$

Distance:

$$d_1 = \sqrt{(11.5 - 4)^2 + (5.5 - 16)^2} = \sqrt{(7.5)^2 + (-10.5)^2} = \sqrt{56.25 + 110.25} = \sqrt{166.5} \approx 12.90 \approx 12.91.$$

**(ii) CU → ALU**

$$C_{CU} = (15, 18), \quad C_{ALU} = (11.5, 5.5)$$

$$d_2 = \sqrt{(11.5 - 15)^2 + (5.5 - 18)^2} = \sqrt{(-3.5)^2 + (-12.5)^2} = \sqrt{12.25 + 156.25} = \sqrt{168.5} \approx 12.98.$$

**(iii) ALU → Cache**

$$C_{ALU} = (11.5, 5.5), \quad C_{Cache} = (15.5, 17)$$

$$d_3 = \sqrt{(15.5 - 11.5)^2 + (17 - 5.5)^2} = \sqrt{(4)^2 + (11.5)^2} = \sqrt{16 + 132.25} = \sqrt{148.25} \approx 12.18.$$

**(iv) RF → Floating Unit**

$$C_{RF} = (4, 16), \quad C_{FU} = (11.5, 8.5)$$

$$d_4 = \sqrt{(11.5 - 4)^2 + (8.5 - 16)^2} = \sqrt{(7.5)^2 + (-7.5)^2} = \sqrt{56.25 + 56.25} = \sqrt{112.5} \approx 10.61.$$

**(v) Cache → Decoder**

$$C_{Cache} = (15.5, 17), \quad C_{Dec} = (6.5, 16.5)$$

$$d_5 = \sqrt{(6.5 - 15.5)^2 + (16.5 - 17)^2} = \sqrt{(-9)^2 + (-0.5)^2} = \sqrt{81 + 0.25} = \sqrt{81.25} \approx 9.01.$$

**(vi) Decoder → Floating Unit**

$$C_{Dec} = (6.5, 16.5), \quad C_{FU} = (11.5, 8.5)$$

$$d_6 = \sqrt{(11.5 - 6.5)^2 + (8.5 - 16.5)^2} = \sqrt{(5)^2 + (-8)^2} = \sqrt{25 + 64} = \sqrt{89} \approx 9.43.$$

**Total wiring distance**

$$W = d_1 + d_2 + d_3 + d_4 + d_5 + d_6 = 12.91 + 12.98 + 12.18 + 10.61 + 9.01 + 9.43 = 67.12 \approx 67.1.$$

## 7.4 Step 3: Bounding box area

We compute:

$$x_{\min} = \min x_i, \quad x_{\max} = \max(x_i + w_i), \quad y_{\min} = \min y_i, \quad y_{\max} = \max(y_i + h_i).$$

From the expanded corners table:

- $x_{\min} = \min\{9, 12, 13, 1, 4, 9\} = 1$
- $x_{\max} = \max\{14, 19, 17, 7, 9, 14\} = 19$
- $y_{\min} = \min\{3, 15, 16, 13, 15, 6\} = 3$
- $y_{\max} = \max\{8, 19, 20, 19, 18, 11\} = 20$

So:

$$A_{\text{bbox}} = (x_{\max} - x_{\min})(y_{\max} - y_{\min}) = (19 - 1)(20 - 3) = 18 \times 17 = 306.$$

## 7.5 Step 4: Overlap count (full pair explanation)

We check overlaps across all  $\binom{6}{2} = 15$  pairs. For  $P1$ , the overlaps are (as in the sample output): **3 overlaps**.

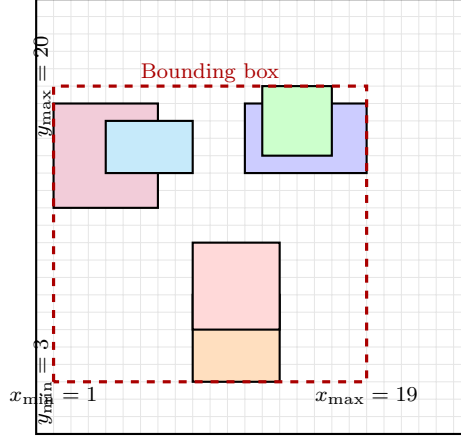


Figure 8: Bounding box for  $P1$ : (1, 3) to (19, 20) giving area 306.

### Which 3 pairs overlap in $P1$ (explicit)

Using rectangle coordinates:

- **ALU** (9, 3)-(14, 8) overlaps with **FU** (9, 6)-(14, 11)
- **Cache** (12, 15)-(19, 19) overlaps with **CU** (13, 16)-(17, 20)
- **RF** (1, 13)-(7, 19) overlaps with **Decoder** (4, 15)-(9, 18)

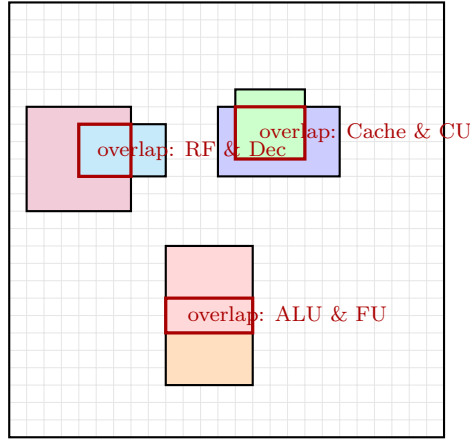


Figure 9:  $P1$  overlap regions (highlighted in red): total overlap count = 3.

### 7.6 Step 5: Total fitness for $P1$ (full calculation)

Given:

$$\alpha = 1000, \quad \beta = 2, \quad \gamma = 1,$$

$$\text{overlap} = 3, \quad W = 67.1, \quad A_{\text{bbox}} = 306.$$

Fitness:

$$F = -(\alpha \cdot \text{overlap} + \beta \cdot W + \gamma \cdot A_{\text{bbox}}) = -(1000 \cdot 3 + 2 \cdot 67.1 + 1 \cdot 306).$$

Compute:

$$1000 \cdot 3 = 3000, \quad 2 \cdot 67.1 = 134.2, \quad 3000 + 134.2 + 306 = 3440.2.$$

So:

$$F \approx -3440.2 \approx -3440.23.$$

## 8 3. Parent Selection

Choose two parents based on **random selection** at each generation. Other popular techniques:

- Tournament selection
- Roulette Wheel selection

## 9 4. Crossover (Single-Point)

Perform **single-point crossover** to create 2 new offspring from each parent pair.

### 9.1 Chromosome gene list

A chromosome has **12 genes**:

$$[x_1, y_1, x_2, y_2, \dots, x_6, y_6],$$

where  $(x_1, y_1)$  is ALU bottom-left,  $(x_2, y_2)$  is Cache bottom-left, etc.

Pick a random split index  $k$  and swap suffixes.

### 9.2 Example: split at $k = 6$ (after 3rd block's $y$ )

$$k = 6 \Rightarrow \text{split after } (x_3, y_3).$$

**Parent gene lists (given)**

$$P1 : (9, 3), (12, 15), (13, 16) \mid (1, 13), (4, 15), (9, 6)$$

$$P2 : (8, 0), (7, 12), (4, 11) \mid (1, 13), (14, 10), (9, 11)$$

**Offspring after crossover**

- Child  $C1$  = first half of  $P1$  + second half of  $P2$ :

$$C1 : (9, 3), (12, 15), (13, 16), (1, 13), (14, 10), (9, 11)$$

- Child  $C2$  = first half of  $P2$  + second half of  $P1$ :

$$C2 : (8, 0), (7, 12), (4, 11), (1, 13), (4, 15), (9, 6)$$

**Evaluate offspring (given results)**

Using the same fitness formula:

- $C1$  fitness  $\approx -2419.88$  (Overlap= 2, Wiring $\approx$  56.94, Area= 306)
- $C2$  fitness  $\approx -4372.33$  (Overlap= 4, Wiring $\approx$  62.66, Area= 247)

**Notice:**

- $C1$  became strong because overlap dropped to 2.
- $C2$  became worse because overlap increased to 4.

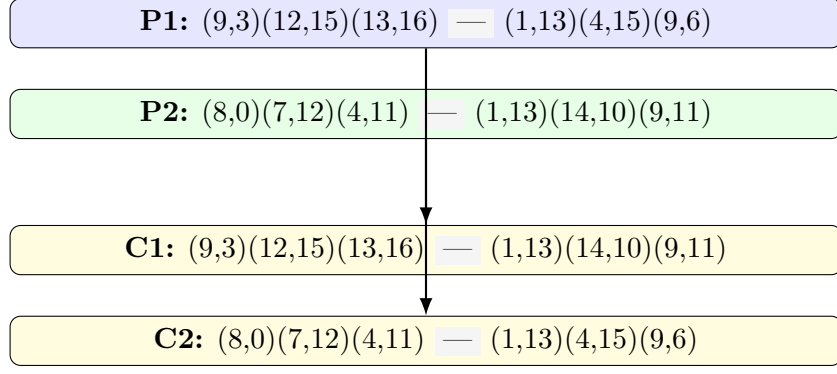


Figure 10: Single-point crossover at  $k = 6$  (swap suffix after the bar).

## 10 5. Mutation

Mutation ensures genetic diversity and prevents the algorithm from getting stuck in local optima.

### 10.1 Mutation strategy

Pick any arbitrary component block and introduce random coordinate changes to that block with low probability (5–10% mutation rate).

### 10.2 Example mutation (given)

Mutate Decoder (block 5) of  $C2$ :

$$(4, 15) \rightarrow (0, 0).$$

So mutated  $C2$  becomes:

$$C2' : (8, 0), (7, 12), (4, 11), (1, 13), (0, 0), (9, 6).$$

Re-evaluate:

$$C2' \text{ fitness} \approx -2415.51 \quad (\text{Overlap} = 2, \text{ Wiring} \approx 74.76, \text{ Area} = 266).$$

## 11 6. New Generation Creation (Elitism)

Population size remains the same for all generations. Apply **elitism** to select new individuals: allow 1 or 2 elites to be carried forward to the next generation, and fill the rest with offspring.

Example (keep 1 elite):

$$\text{Elite} = P5 \text{ (fitness } -2429.00), \quad \text{NewPop} = \{P5\}, \text{ need 5 more individuals.}$$

## 12 7. GA Loop

Run genetic algorithm iterations until:

- best fitness (plateau) is achieved, **or**
- maximum number of **15 iterations** is reached.

## 13 8. Required Output

When the algorithm halts, output:

- best **total fitness value**,
- corresponding **total wiring length**,
- **total bounding box area**,
- **total overlap count**,
- decoded best chromosome (bottom-left coordinates of all blocks).

## Algorithm: GeneticAlgorithm\_Floorplanning\_Task1

**Input** : popSize = 6, mutationRate  $\in [0.05, 0.10]$ ,  $\alpha = 1000, \beta = 2, \gamma = 1$ ,  
maxGenerations = 15  
**Output**: Best placement solution found

Population  $\leftarrow$  InitializeRandomPopulation(popSize)  
**foreach** *chromosome G in Population* **do**  
    |  $G.\text{fitness} \leftarrow \text{Fitness}(G)$   
**end**  
generation  $\leftarrow 0$   
bestSolution  $\leftarrow \text{BestIndividual}(\text{Population})$   
**while** *generation < maxGenerations* **do**  
    // Elitism  
    NewPopulation  $\leftarrow \text{TopElite}(\text{Population}, 1)$   
    **while** *size(NewPopulation) < popSize* **do**  
        // Parent Selection (Random)  
        parent1, parent2  $\leftarrow \text{RandomSelect}(\text{Population})$   
        // Single-Point Crossover  
        child1, child2  $\leftarrow \text{SinglePointCrossover}(\text{parent1}, \text{parent2})$   
        // Mutation (5--10%)  
        child1  $\leftarrow \text{Mutate}(\text{child1}, \text{mutationRate})$   
        child2  $\leftarrow \text{Mutate}(\text{child2}, \text{mutationRate})$   
        // Evaluate offspring  
        child1.fitness  $\leftarrow \text{Fitness}(\text{child1})$   
        child2.fitness  $\leftarrow \text{Fitness}(\text{child2})$   
        Add child1 to NewPopulation if space available  
        Add child2 to NewPopulation if space available  
    **end**  
    Population  $\leftarrow \text{NewPopulation}$   
    currentBest  $\leftarrow \text{BestIndividual}(\text{Population})$   
    **if** *currentBest.fitness > bestSolution.fitness* **then**  
        | bestSolution  $\leftarrow \text{currentBest}$   
    **end**  
    generation  $\leftarrow \text{generation} + 1$   
**end**  
**return** bestSolution

**Algorithm 1:** GeneticAlgorithm\_Floorplanning\_Task1

## 14 Task 2 (Two-point crossover)

Incorporate **two-point crossover** into Step 4:

- Randomly select two parents.
- Choose two random crossover points  $k_1 < k_2$ .
- Swap the middle segment  $(k_1 + 1, \dots, k_2)$  to generate two children.
- Ensure the second crossover point always comes after the first.

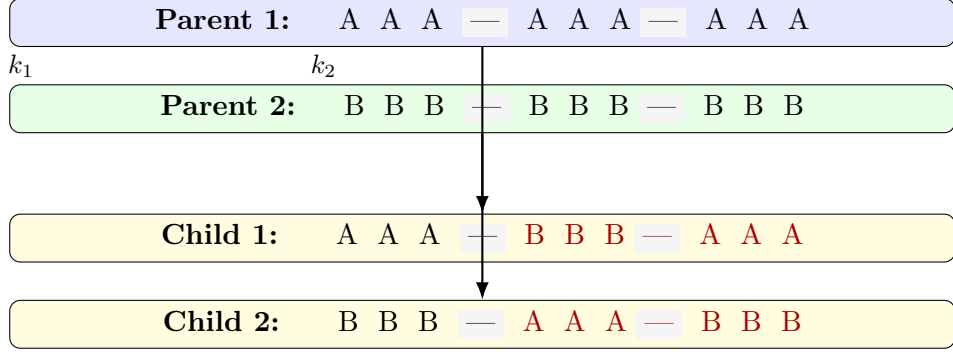


Figure 11: Two-point crossover: swap the middle segment between  $k_1$  and  $k_2$ .

## Sample Input and Output (as listed)

Initial population (same as earlier):

$P1 \rightarrow (9, 3), (12, 15), (13, 16), (1, 13), (4, 15), (9, 6)$   
 $P2 \rightarrow (8, 0), (7, 12), (4, 11), (1, 13), (14, 10), (9, 11)$   
 $P3 \rightarrow (6, 5), (12, 9), (9, 7), (8, 6), (2, 7), (3, 1)$   
 $P4 \rightarrow (3, 11), (11, 12), (14, 11), (6, 10), (3, 11), (3, 0)$   
 $P5 \rightarrow (10, 12), (8, 16), (10, 4), (13, 6), (6, 0), (3, 7)$   
 $P6 \rightarrow (0, 2), (0, 0), (14, 12), (4, 5), (12, 4), (3, 10)$

For  $P1$ :

- Pairwise block overlap count = 3
- Total wiring distance (center-to-center) of specified pairs:

$$12.91 + 12.98 + 12.18 + 10.61 + 9.01 + 9.43 = 67.1$$

- Total bounding box area:

$$(x_{\max} - x_{\min})(y_{\max} - y_{\min}) = (19 - 1)(20 - 3) = 306$$

- Total fitness:

$$F = -(1000 \cdot 3) - (2 \cdot 67.1) - 306 = -3440.23$$